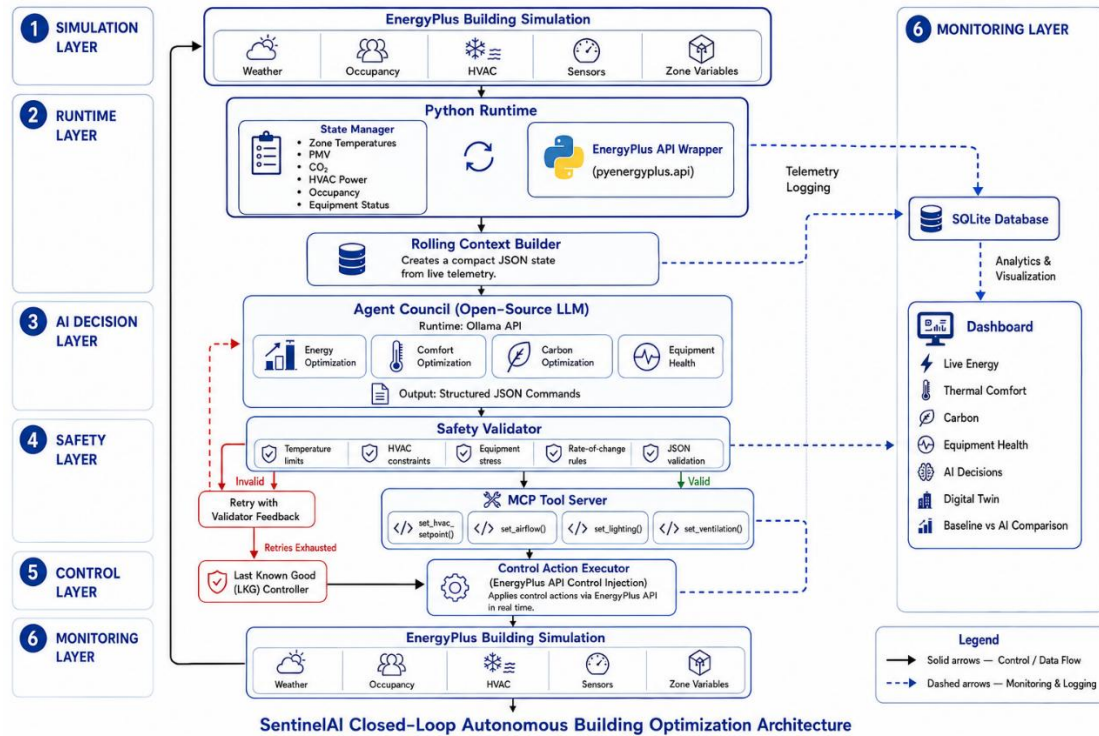


SentinelAI: System Architecture & Technical Approach

Soumyadip Chakrabarti

23BAI1265

System Architecture Diagram



1. Tool-Calling Architecture & Agent Orchestration

SentinelAI employs a deterministic **Model Context Protocol (MCP)** architecture to bridge the gap between probabilistic Large Language Models (LLMs) and the strict physical constraints of the EnergyPlus physics engine.

- **Agent Council Orchestrator:** Instead of using raw unstructured generation, the LLM is prompted as a multi-objective "Agent Council". It outputs a strict JSON payload mapping directly to MCP tool signatures (e.g., `set_hvac_setpoint`, `set_airflow`).
- **Safety Validator Interceptor:** Before any LLM output touches the building, a 9-rule Safety Validator intercepts the proposed JSON. It checks physical bounds (e.g., setpoints between 18-26°C), rate limits (max 1.5°C delta per hour), and equipment stress bounds. If a rule is violated, it intercepts the tool call and forces the LLM to self-correct using targeted error feedback.
- **EnergyPlus C-API Bridge:** Once validated, the JSON payloads are converted into memory-address pointers and injected directly into the EnergyPlus C engine via `pyenergyplus.api` at every 15-minute zone timestep.

2. Prompt Engineering Strategies

To maintain consistent reasoning across multiple optimization objectives while minimizing hallucinations, SentinelAI uses a unified structured prompting strategy.

Rather than deploying four separate agent prompts (which multiplies latency and token costs), the prompt forces the LLM to write distinct reasoning chains for each objective *before* emitting the final actuator tool calls. The prompt instructs the model to explicitly evaluate energy efficiency, thermal comfort, carbon impact, and equipment health before producing a structured JSON action. This encourages consistent multi-objective reasoning while keeping the output deterministic.

3. Prompt Latency Management

In a live HVAC control system, latency is a critical safety parameter. We manage latency through three layers:

1. **Streaming Response Processing:** The system supports chunked streaming from the LLM. Instead of waiting for the full response, a custom regex parser aggregates the stream and extracts the JSON tool call block in real-time.
2. **Context Minimization:** We strictly forbid sending raw simulation logs to the LLM. The prompt consists only of a highly-dense JSON `State` object (occupancy, PMV, temps) and a short list of safety rule violations if it is in a retry state.
3. **Deterministic Fallback (LKG):** If the LLM times out (e.g., >10 seconds), experiences API failure, or fails safety validation 3 times in a row, the system instantly halts the LLM pipeline and falls back to a deterministic Last-Known-Good (LKG) rule-based engine, ensuring zero downtime in physical building operations.

4. Handling Lengthy Simulation Logs

EnergyPlus generates massive, verbose text logs (`.eso`, `.err`, `.audit`) that easily exceed million-token context windows if fed natively to an LLM.

- **C-API Polling (Zero-Log Execution):** SentinelAI completely bypasses text-log parsing. We use the EnergyPlus Python API to read variables directly from C memory pointers at runtime.
- **Rolling Context Builder:** Raw telemetry (e.g., fan power watts, room temperatures) is fed into a `ContextBuilder` which summarizes the data into a 10-step rolling average summary.
- **SQLite Decoupling:** The full, lengthy simulation logs are written atomically to a structured SQLite database (`sentinel_ai.db`). The LLM never sees this database. Instead, the Live Dashboard queries SQLite for quantitative plotting, isolating the LLM entirely from "Big Data" bloat.

Conclusion

SentinelAI demonstrates a complete closed-loop autonomous building management workflow by combining EnergyPlus, an LLM-based Agent Council, MCP tool execution, safety validation, and real-time telemetry. The architecture prioritizes deterministic tool execution, low-latency inference, and robust operation while maintaining occupant comfort and reducing energy consumption.